



ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba



UNIVERSIDAD DE CÓRDOBA

Plataformas Big Data

Curso 2026/2027

Boletín de Ejercicios

Prof: Dr. Héctor Martínez Pérez

Dpto. Ingeniería Electrónica y Computadores

Arquitectura de Computadores, Electrónica y Tecnología Electrónica

2º Cuatrimestre

Código: 283007

EJERCICIO 1

Manejo para datos de sensores IoT

[Complejidad Computacional] - [Optimización]

Disponemos de un vector con una capacidad de 1 millón de elementos en coma flotante, el cual representa valores recogidos por un sensor durante un día. Antes de llevar a cabo su estudio, hay que aplicarles una serie de correcciones a los valores almacenados. Para ello, inicialmente se generará un vector en memoria de 1 millón de elementos de tipo float, y se inicializará con valores aleatorios entre 0 y 1 (simulando medidas de humedad). A partir de este punto, el alumno llevará a cabo una normalización Min-Max, para ello, se calcularán el valor máximo XMAX y el valor mínimo XMIN y el valor normalizado se obtendrá mediante:

$$X' = (X - XMIN) / (XMAX - XMIN)$$

Ejercicio 1.1 - El alumno desarrollará en un tiempo máximo de 30 minutos un programa Python capaz de llevar a cabo esta tarea. El código se temporizará mediante la rutina `time.time()`

Ejercicio 1.2 - De igual manera que el ejercicio anterior, el alumno dispondrá de un tiempo total de 30 minutos para llevar a cabo un programa en C que lleve a cabo esta misma tarea. El código se temporizará mediante la rutina:

```
double dclock() {  
    struct timeval  tv;  
  
    gettimeofday( &tv, NULL );  
  
    return (double) (tv.tv_sec + tv.tv_usec*1.0e-6);  
}
```

Ejercicio 1.3 - Comente los resultados temporales obtenidos con el resto del alumnado.

PLATAFORMAS BIG DATA

3º de Ingeniería Informática

EJERCICIO 2

Procesamiento estadísticas Netflix

[Complejidad Computacional] - [Optimización]

Una plataforma de streaming como Netflix recalcula constantemente el tiempo de visionado de los clientes para las recomendaciones personalizadas. Para representar esta información, imagine que tenemos un vector multidimensional, donde el número de filas es el número de usuarios de la plataforma y el número de columnas es el número de películas disponibles para los usuarios.

Según diversas fuentes de internet, actualmente existen unos 9.000 títulos disponibles en Netflix para sus usuarios. Respecto a la cifra de usuarios, la cantidad ronda los 325 millones.

Ejercicio 2.1 - Imagine que queremos representar estos datos con una matriz, eligiendo el tipo de dato entero (4 bytes). ¿Qué cantidad de memoria (ya sea en disco duro o en memoria RAM) necesitaríamos para representarlo?

Ejercicio 2.2 - Como se trata de una cifra muy elevada, se escalará el problema. Supongamos que el número de usuarios activos es de 30.000; esto conlleva una cantidad aproximada de datos de 1 GB en este caso. En la plataforma Moodle, el alumno dispone de un pequeño programa Python "**generador_netflix.py**" que generará un pequeño fichero CSV de 1 GB (aprox.) de datos, donde cada fila es un cliente y cada columna las horas vistas de cada serie por el mismo. Una vez descargado y ejecutado mediante el comando:

```
$ python generador_netflix.py
```

El alumno deberá procesar el propio fichero mediante un programa en C, midiendo el tiempo total de su procesamiento. La información que se busca es:

- Media de tiempo de visionado por cliente.
- Contenido más visto por cada cliente.
- Contenido más visto de forma global.

Ejercicio 2.3 - Una vez realizada la primera implementación en Python, el alumno deberá llevar a cabo esta misma tarea desde C.

Ejercicio 2.4 - El tamaño que alcanzaría este fichero para el caso real es tan elevado que no cabría en la memoria RAM de un servidor. Por este motivo, se pide al alumno que modifique su implementación para procesarlo por bloques, técnica usada comúnmente en este tipo de escenarios.

EJERCICIO 3

Procesamiento de ficheros log en servidores

[Complejidad Computacional] - [Optimización]

Para el siguiente caso de estudio, el alumno tendrá a su disposición el programa Python llamado “**generador_server.py**”. Este programa creará un fichero de logs de un servidor. Antes de comenzar, el alumno ejecutará el script mediante el comando:

```
$ python generador_server.py
```

El fichero generado contendrá por cada línea en el siguiente orden: *marca de tiempo de la conexión, ip, método de conexión, URL, código http, bytes transferidos y cliente*.

Ejercicio 3.1- Se necesita conocer, en primer lugar, la dirección IP desde la cual se ha llevado a cabo un mayor tráfico y cuál es la diferencia entre la media de datos que se han transferido en el servidor. Gracias a este estudio, la empresa será capaz de detectar conexiones potencialmente peligrosas.

El alumno deberá considerar que la memoria de su sistema es limitada y únicamente se podrá reservar un total de 64 MB de memoria para llevar a cabo este procesamiento.

EJERCICIO 4

Procesamiento de secuencias de ADN

[Complejidad Computacional] - [Optimización] - [OpenMP]

Uno de los campos donde se aplica la ciencia de datos y se hace uso de plataformas de big data a gran escala es el campo de la genómica. Para el estudio de secuencias de ADN/ARN, en primer lugar se secuencian el genoma humano, creando ficheros de tamaños muy grandes (cientos de gigas), repletos de secuencias de ADN que únicamente contienen los caracteres: A,C,G y T (nucleótidos). Estas lecturas hay que encontrarlas en un genoma de referencia de la especie que se trate, por ejemplo, el genoma de referencia del Homo sapiens ocupa aproximadamente 3 GB. Para su procesamiento, hay que encontrar cada una de estas secuencias dentro del genoma de referencia y detectar los fallos.

Para este problema, el alumno tendrá un escenario similar, pero lógicamente simplificado. En primer lugar, el alumno ejecutará el programa “**generador_adn.py**”, disponible en Moodle:

```
$ python generador_adn.py
```

Posteriormente, el alumno dispondrá de una secuencia de referencia, que le mostrará el programa por pantalla, y un fichero de 1 GB con lecturas de ADN de 50 nucleótidos, y algunas de ellas mutadas. Entonces, para llevar a cabo el ejercicio, el alumnado deberá desarrollar un programa en C, lo más óptimo posible, que compare cada una de las lecturas que contiene el fichero con la lectura de referencia. Se deberá generar un fichero de salida que contenga las mutaciones. Para ello, se anotará en primer lugar el identificador de lectura (posición que ocupa en el fichero de entrada) y, separado por un tabulador, la lista de errores que contiene la lectura (si la lectura no contiene errores, no será almacenada en el fichero de salida). Los errores se representarán de la siguiente manera: Primero posición del error, pegado a él, el nucleótido encontrado, y finalmente el nucleótido de referencia, tal que así:

Referencia -> ATACTTTGGT

Lectura -> ATACTATGAT

Salida -> **SEQ0: 6AT, 9AG**

No es necesario colocar “y” en el último error de la lista, únicamente la separación con el carácter “,”.

Ejercicio 4.1 - Esto es un procesamiento crítico en el campo de la genómica; el alumno debe ser consciente de que **0.1s** menos por **secuencia**, si se dispone de **1 millón** de secuencias por **muestra**, es un ahorro de tiempo de **1 día por muestra**! Dicho esto, el alumno desarrollará de la forma más óptima posible este procesamiento. Por último, el alumno temporizará su procesamiento.

Ejercicio 4.2 - Los alumnos intercambiarán entre ellos sus implementaciones y revisarán las de sus compañeros, indicando qué optimizaciones aplicaron ellos en esa implementación. Seguidamente, cada alumno recuperará su propia implementación y comprobará las modificaciones.

Ejercicio 4.3 - Cada alumno preguntará a su modelo de inteligencia artificial favorito el planteamiento del siguiente problema y le pedirá que le genere una solución, la integrará y temporizará contrastando los resultados con su propia implementación.

Ejercicio 4.4 - [OpenMP] - Cuando el alumno haya aprendido las diferentes técnicas de paralelismo, desarrollará una versión paralela haciendo uso de todos los cores del sistema; además, comprobará con el comando htop el funcionamiento al máximo de todos los procesadores del sistema. Además, el alumno creará una tabla con la escalabilidad del problema (tiempo de procesamiento por número de hilos utilizados).

Nota: El tamaño del fichero es orientativo; se busca que sea un procesamiento costoso, pero en función de las características de la máquina puede ser excesivo. En caso de tardar mucho tiempo su procesamiento reduzca el tamaño del fichero a un tamaño donde el tiempo su procesamiento ronde el minuto.

EJERCICIO 5

Aceleración del procesamiento de imágenes

[Complejidad Computacional] - [Optimización] - [SIMD]

En el campo de la visión por computador, el procesamiento de imágenes es una tarea cotidiana. Una vez se obtiene una imagen, se aplican filtros, transformaciones, etc. Para así llevar a cabo su procesamiento. En este campo es muy común el uso de la programación sobre GPU, no obstante, vamos a utilizar otros tipos de técnicas de aceleración del procesamiento orientadas a CPU.

Un escenario real, aparte del campo de las redes convolucionales, son las fábricas de vehículos. En ellas se lleva a cabo un estudio a fondo, mediante cámaras de alta resolución, sobre la pintura de la carrocería de los vehículos. Para ello, múltiples cámaras generan un flujo constante de imágenes para su estudio y poder así detectar fallos en la pintura de la carrocería.

Muchas empresas de este tipo recurren a expertos en ciencia de datos o computación de altas prestaciones para la aceleración del procesamiento de estas imágenes. El alumno debe tener presente que en la época en que vivimos y con la tecnología disponible, cada vez se obtiene una mayor cantidad de datos y el tiempo esperado para su procesamiento es mínimo. Por este motivo, se plantea el siguiente ejercicio.

La empresa nos solicita que aceleremos el procesamiento de las imágenes que producen (tamaño 5120px x 5120px). Las imágenes son RGB y contienen un total de 3 canales (RGB). El almacenamiento de estas imágenes se lleva a cabo en este formato, por lo tanto, dispondremos de un puntero a un vector con la siguiente información:

PX0-R, PX0-G, PX0-B, PX1-R, PX1-G, PX1-B, ...

Ejercicio 5.1 - Desarrolle un programa en C que lleve a cabo las reservas de memoria pertinentes e inicialice los valores de cada píxel con un valor aleatorio entre 0 y 255. Para la imagen, el procesamiento que llevará a cabo es la aplicación de un filtro 1x1. Para ello se deberá implementar una de las operaciones más utilizadas en redes convolucionales, la convolución. La Figura 1 muestra un ejemplo de este algoritmo para un filtro 4x4.

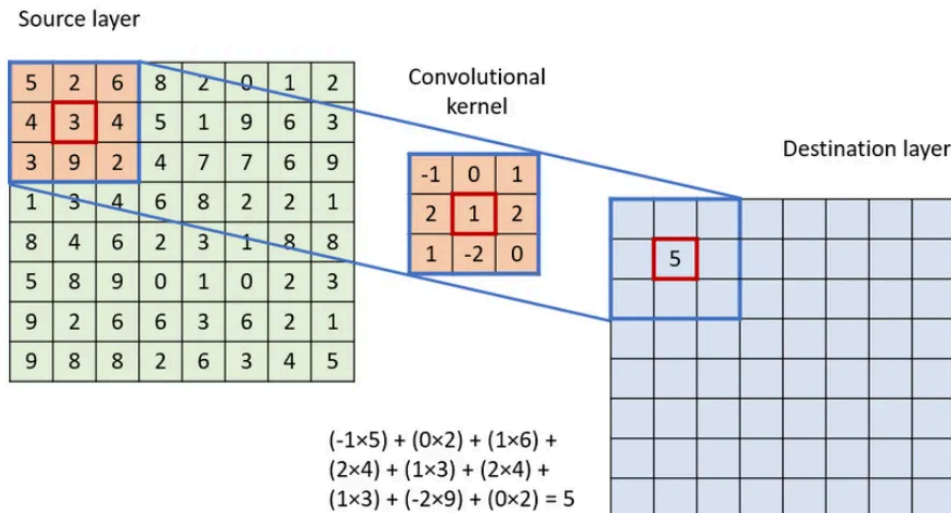


Figura 1. Diagrama del algoritmo de la convolución.

Fuente: <https://medium.com/@abhishekjainindore24/all-about-convolutions-kernels-features-in-cnn-c656616390a1>

El filtro 1x1 (1 píxel) irá recorriendo la imagen de izquierda a derecha y de arriba abajo, una vez aplicado el filtro, se moverá una posición hacia la derecha para el cálculo del siguiente píxel. En este caso, al ser un filtro de 1 píxel, únicamente hay que multiplicar todos los elementos de la matriz por este valor. El filtro contendrá el valor de 2 y el valor de cada píxel se calculará con $C[p] += C[p] \times F[0]$.

Ejercicio 5.2 - Optimice el algoritmo desarrollado con las técnicas aprendidas de vectorización. Para esto, en primer lugar deberá temporizar cuánto tarda su implementación anterior y la nueva. Por otro lado, tenga en cuenta la arquitectura en la que está ejecutando el algoritmo (Intel, AMD, ARM, etc.); en función de ella, deberá consultar el manual de referencia de instrucciones intrínsecas para esta arquitectura en concreto.

INTEL/AMD:

<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>

ARM:

<https://developer.arm.com/architectures/instruction-sets/intrinsics/>

Nota: Tenga en cuenta que se busca el mayor rendimiento posible, se busca un algoritmo altamente optimizado y acorde a Plataformas Big Data y Computación de Altas Prestaciones.

EJERCICIO 6

Detección de piezas defectuosas

[Complejidad Computacional] - [Optimización] - [OpenMP]

El objetivo principal de este problema es detectar defectos en una pieza. Para ello, la empresa dispone de un sistema de inspección automático de piezas; cada pieza se representa mediante una imagen binaria almacenada en un fichero de texto.

La imagen está formada únicamente por los valores:

- 0: píxel negro.
- 1: píxel blanco.

La pieza se representa mediante píxeles blancos sobre un fondo negro.

Una pieza sin defectos tiene únicamente píxeles blancos en su interior. Un defecto se representa mediante uno o varios píxeles negros completamente rodeados por píxeles blancos.

Por ejemplo, la siguiente imagen representa una pieza sin defectos:

Sin defectos

```
000000000000
000011110000
000111111000
000111111000
000011110000
000000000000
```

Contiene 1 defecto

```
000000000000
000011110000
000111111000
000110111000
000011110000
000000000000
```

El objetivo es desarrollar un programa en **C** que lea la imagen desde un fichero y determine si la pieza contiene defectos.

La imagen la podrá generar el alumno mediante el programa llamado **"generador_piezas.py"** y le pasará el tamaño de la pieza como parámetro, alto y ancho.

Para comenzar se generará una pieza de 100 x 100 de la siguiente manera:

```
$ python generador_piezas.py 100 100
```

La primera línea del fichero contiene la información de la pieza; en ella están el tamaño y una lista con los defectos que contiene (su posición), para su posterior validación.

Una vez generada la pieza, el alumno debe de tener en cuenta que un defecto se considera cuando:

1. El píxel actual es metro (0)
2. No pertenece al borde de la imagen.
3. Sus cuatro vecinos inmediatos son blancos (1):

```
1
1 0 1
1
```

Ejercicio 6.1 - El primer ejercicio consiste en una implementación básica donde el alumno leerá de un fichero la pieza (100x100), la cargará en memoria y la procesará, mostrando una lista con las posiciones del error que contiene de la siguiente manera:

IMAGEN: 100 X 100

DEFECTOS ENCONTRADOS:

* 45, 48

* 106,230

DIAGNÓSTICO: PIEZA DEFECTUOSA

Ejercicio 6.2 - Una vez desarrollado el programa, y comprobado su correcto funcionamiento, es la hora de llevarlo a plataformas Big Data, para esto, se generarán imágenes de diferentes tamaños: 500 x 500, 1000 x 1000, 2000 x 2000, 5000 x 5000, etc. En primer lugar, el alumno temporizará el tiempo de procesamiento de cada una de estas imágenes comprobando que el resultado sea el correcto.

Ejercicio 6.3 - En este paso, el alumno estudiará su implementación inicial y comprobará si es una implementación óptima, aplicará las optimizaciones oportunas y llevará a cabo una nueva implementación paralela. El alumno deberá reflexionar sobre las diferentes opciones de paralelización y elegir la óptima (discutir en clase).

PLATAFORMAS BIG DATA

3º de Ingeniería Informática

EJERCICIO 7

Virtualización de servidores

[Virtualización]

En la teoría se ha visto que virtualizar significa crear una capa de abstracción sobre los recursos de un ordenador. Partiendo de esta premisa, existen dos maneras de hacerlo: mediante máquinas virtuales (que emulan un ordenador completo) o contenedores (que aíslan solo un proceso y su entorno, reutilizando el kernel del sistema anfitrión).

Para el desarrollo de esta práctica será necesaria la instalación de VirtualBox y Docker Desktop. Para ello, puede seguir las indicaciones durante el guion del ejercicio.

Parte 1 - VirtualBox

En primer lugar instale VirtualBox de la manera que desee o mediante el siguiente comando:

```
$ sudo apt-get install -y virtualbox
```

Paso 1.1 - Descarga del Sistema Operativo

El primer paso consistirá en descargar una ISO de una distribución Linux que elija el alumno (se recomienda Alpine Linux debido a sus bajos requerimientos).

<https://alpinelinux.org/downloads/>

Nota: Tenga en cuenta la arquitectura de su PC a la hora de descargar la ISO del sistema operativo.

Paso 1.2 - Crear la máquina virtual

1. Abre VirtualBox y pulsa el botón "Nueva".
2. Ponga el nombre "VM-Practica" y seleccione como tipo "Linux" y versión "Other Linux (64-bit)".
3. Asígnele **512 MB** (siempre y cuando le quede suficiente memoria para su PC).
4. Cree un disco duro virtual nuevo de tipo VDI, de tamaño fijo, de **8 GB**.
5. En la configuración de la VM, vaya a "Almacenamiento" y monte la imagen .iso de Alpine (o la elegida por el alumno) que descargó como unidad de CD/DVD.
6. Asigne 1 CPU virtual (vCPU) en la pestaña "Sistema" → "Procesador".

***Nota:** Fíjese en que, antes incluso de arrancar, ya hemos tenido que decidir cuánta RAM, disco y CPU 'reservar' para esta máquina en exclusiva. Esto es lo que denominamos 'asignación rígida de recursos': el hipervisor reserva esos recursos para la VM aunque no los esté usando.*

Pregunta 1. Mientras la VM esté encendida con 512 MB de RAM asignados, ¿esa memoria seguirá estando disponible para el resto de programas de su ordenador (el navegador, Word, etc.)? Justifique su respuesta.

Paso 1.3 - Crear la máquina virtual

1. Antes de pulsar "Iniciar", prepare un cronómetro (el del móvil sirve).
2. Pulse "Iniciar" en VirtualBox y arranque el cronómetro en el mismo instante.
3. Para el cronómetro en el momento en que vea el símbolo del sistema (prompt) de Alpine Linux (o el sistema operativo elegido), listo para escribir comandos.
4. Anote el tiempo total de arranque en la tabla siguiente (lo usaremos más adelante para comparar).

Paso 1.4 – Observar el disco de la VM

1. Apague la máquina virtual (menú "Máquina" → "Apagar", o simplemente cierre la ventana y elija "Apagar la máquina").
2. Busque en su explorador de archivos la carpeta donde VirtualBox guarda las VMs (normalmente 'VirtualBox VMs' en su carpeta de usuario) y localice el archivo .vdi de su VM-Practica.
3. Anote su tamaño en disco.

Pregunta 2. Compare el tamaño del archivo .iso original de la distribución Linux con el tamaño del disco .vdi de la VM ya creada. ¿A qué se debe la diferencia?

Parte 2 - Docker

Para la instalación de Docker puede seguir las siguientes instrucciones:

```
$ sudo apt-get install -y docker.io docker-compose
```

```
$ sudo systemctl enable --now docker
```

```
$ sudo usermod -aG docker $USER
```

Para finalizar, reinicie el sistema y ejecute una vez arrancado de nuevo el sistema operativo:

```
$ docker --version
```

Paso 2.1 – Primer contenedor

En primer lugar ejecutaremos el comando:

```
$ docker run hello-world
```

Este comando solicita a Docker que ejecute un contenedor a partir de la imagen *'hello-world'*. Si es la primera vez que se usa, *Docker* la descargará automáticamente de Docker Hub antes de ejecutarla. Verá una serie de mensajes detallados por Docker de las operaciones que ha llevado a cabo internamente.

Paso 2.2 – Contenedor para un servidor web

Ahora levantaremos un servidor web real dentro de un contenedor. Antes de ejecutar el comando, prepare el cronómetro como hizo con la máquina virtual.

```
$ docker run -d -p 8080:80 --name mi_web nginx
```

- `docker run` → Crea y arranca un contenedor nuevo.
- `-d` → Se lanza en segundo plano (*detached*).
- `-p 8080:80` → Conecta el puerto 8080 del PC con el puerto 80 interno al contenedor, donde escucha el servidor web.
- `--name mi_web` → Nombre del contenedor.
- `nginx` → Nombre de la imagen que deseamos ejecutar (en este caso se trata de un servidor web ampliamente utilizado)

Pare el cronómetro en cuanto termine la ejecución del comando y devuelva el identificador del contenedor.

Abra el navegador y visite la página: <http://localhost:8080>. Deberá ver la página de bienvenida de Nginx.

Pregunta 3. ¿Cuánto ha tardado en estar disponible el servidor web, comparado con el tiempo que tardó en arrancar la máquina virtual de la parte 1? ¿A qué cree que se debe esta diferencia de tiempo?

Paso 2.3 – Recursos del sistema

Escriba en un terminal:

```
$ docker ps
```

Este comando lista todos los contenedores que están en ejecución en este instante en el sistema. Ahora compruebe los recursos que está consumiendo realmente:

```
$ docker stats --no-stream
```

Fíjese en las columnas CPU% y MEM USAGE/LIMIT.

Pregunta 4. Compare el consumo real de memoria de su contenedor Nginx con los 512 MB que reservó forzosamente para la máquina virtual. ¿Qué ventaja práctica ve usted en un servidor donde se alojan multitud de aplicaciones?

Al igual que VirtualBox, nos permite parar y arrancar contenedores, para ello tenemos disponibles los comandos:

```
$ docker stop mi_web
```

```
$ docker start mi_web
```

Puede ejecutarlos para observar la velocidad de parada y arranque de los contenedores Docker.

Antes de continuar con los siguientes pasos, eliminaremos el contenedor (no la imagen); esta se mantiene en su ordenador para la próxima vez. Dicho esto, ejecute:

```
$ docker stop mi_web
```

```
$ docker rm mi_web
```

Paso 2.4 – Construir una imagen propia

Hasta ahora ha usado imágenes preconstruidas (nginx, hello-world). En este paso creará su propia imagen a partir de un Dockerfile.

En primer lugar, se creará una carpeta nueva en su PC llamada practica-docker. Seguidamente, en su interior, crearemos un archivo index.html con el siguiente contenido:

```
<html>
<body>
<h1> Página de [nombre del alumno] - versión 1.0 </h1>
</body>
</html>
```

En esta misma carpeta cree un archivo de texto llamado exactamente Dockerfile (sin extensión) con este contenido:

```
FROM nginx:alpine
```

```
COPY index.html /usr/share/nginx/html/index.html
```

- FROM → Indica que el contenedor partirá de la imagen oficial de Nginx versión Alpine.
- COPY → Copia el archivo index.html dentro del contenedor en la carpeta donde Nginx busca la página web principal del servidor.

Desde terminal, entre en la carpeta creada y construya la imagen

```
$ docker build -t mi-pagina:1.0 .
```

- docker run → Construye una imagen a partir de un Dockerfile.
- -t mi-pagina:1.0 → Fija el nombre 'mi-pagina' y la etiqueta (versión) a 1.0.
- . → El punto final indica a Docker que debe usar el Dockerfile disponible en el propio directorio.
-

Compruebe que su imagen ha sido creada

```
$ docker images
```

Finalmente ejecute su imagen:

```
$ docker run -d -p 8081:80 --name mi_pagina_v1 mi-pagina:1.0
```

De nuevo, visite <http://localhost:8081> y compruebe que ve la página creada.

Ahora edite el fichero index.html y cambie el texto a 'versión 2.0'. A continuación construya una nueva imagen SIN sobrescribir la anterior.

```
$ docker build -t mi-pagina:2.0 .
```

```
$ docker run -d -p 8082:80 --name mi_pagina_v2 mi-pagina:2.0
```

Ahora visite las dos direcciones y compare:

- <http://localhost:8081> → Debería seguir mostrando versión 1.0
- <http://localhost:8082> → Debería mostrar la nueva versión 2.0

Por último, ejecute de nuevo:

```
$ docker images
```

Pregunta 5. Describa la diferencia entre '*imagen*' y '*contenedor*'.

Paso 2.5 – Redes: Comunicar dos contenedores

Somos conocedores de la desventaja de los contenedores, la persistencia de los datos. Para comprobar esto, siga los siguientes pasos:

En primer lugar montaremos una base de datos simple en modo interactivo

```
$ docker run -d - --name contenedor_nuevo redis
```

Ahora desde otro contenedor cualquiera, intente hacer ping a este contenedor nuevo por su nombre:

```
$ docker run - --rm alpine ping -c 2 contenedor_nuevo
```

Pregunta 6. ¿Qué mensaje de error obtiene? ¿Por qué cree que un contenedor no puede encontrar a otro por su nombre?

Siga ahora los siguientes pasos:

```
$ docker network create mired
```

```
$ docker rm -f contenedor_nuevo
```

```
$ docker run -d - --name contenedor_nuevo - --network mired redis
```

```
$ docker run - -rm - -network mired alpine ping -c 2 contenedor_nuevo
```

Pregunta 7. Ahora debería de haber funcionado el comando ping. Reflexione sobre qué ha cambiado respecto al anterior escenario.

Para finalizar el ejercicio eliminaremos los últimos pasos ejecutados para dejar la máquina limpia:

```
$ docker rm -f contenedor_nuevo
```

```
$ docker network rm mired
```

EJERCICIO 8

Análisis y paralelización de una aplicación industrial

[Complejidad Computacional] - [Análisis] - [Optimización] - [OpenMP] - [MPI]

Una empresa industrial dispone de una máquina de fabricación cuyo rendimiento depende de diversos parámetros de funcionamiento. Entre otros, los ingenieros de la empresa pueden fijar:

- Temperatura de funcionamiento
- Presión de trabajo
- Velocidad de producción

La empresa dispone de un software que permite evaluar el coste asociado a una configuración específica de la máquina. El objetivo de este software es determinar la configuración que minimiza el coste de fabricación. Para ello, el programa actual realiza una búsqueda exhaustiva: prueba todas las combinaciones posibles de temperatura, presión y velocidad y calcula el coste de cada una.

El software actual funciona correctamente, pero el número de configuraciones que debe analizar ha aumentado considerablemente, haciendo su ejecución demasiado lenta.

La empresa dispone de un clúster formado por varios nodos de cómputo del que le gustaría sacar rendimiento y por ello necesita una versión paralela y óptima de la aplicación.

Ejercicio 8.1 - El alumno tiene a su disposición la aplicación llamada "*industrial_op.c*", la cual compilará mediante el siguiente comando:

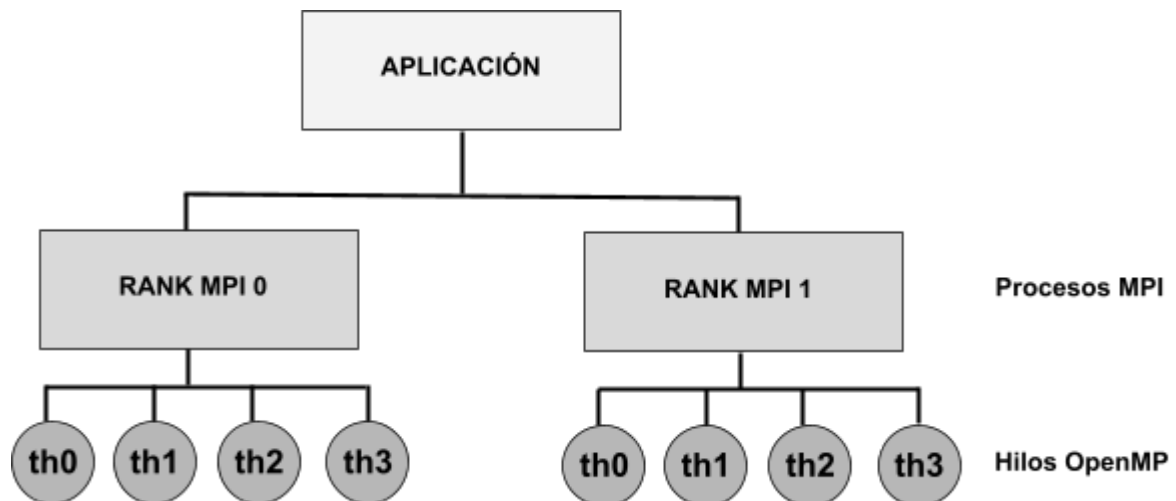
```
$ gcc -O3 -Wall industrial_op.c -o industrial_op -lm
```

En primer lugar, se llevará a cabo un análisis de la aplicación detectando posibles cuellos de botella e intentando optimizar las partes que el propio alumno vea convenientes.

Ejercicio 8.2 - En el segundo lugar, se llevará a cabo una paralelización con base en MPI, repartiendo el trabajo entre los diferentes nodos de un clúster. Se considerará que se dispone de 2 nodos; por lo tanto, el número máximo de procesos MPI con los que se lanzará la aplicación será 2.

Ejercicio 8.3 - En tercer lugar, cada nodo dispone de 4 núcleos, por lo tanto, una vez repartido el trabajo entre los dos nodos del clúster, cada nodo paralelizará el problema con hilos OpenMP. De igual manera, el máximo número de hilos en el cual se dividirá el trabajo será de 4.

Esquema de paralelización de la aplicación.



EJERCICIO 9

Sistema recomendaciones Amazon

[Complejidad Computacional] - [Análisis] - [Optimización] - [CUDA]

Amazon dispone de un catálogo formado por millones de productos. Para mejorar las recomendaciones a sus clientes, cada producto se representa mediante un vector numérico de características. Estas características pueden representar, por ejemplo:

- Precio.
- Categoría.
- Características físicas.
- Valoraciones.
- Popularidad.
- Características obtenidas mediante técnicas de aprendizaje automático.
- Otras propiedades numéricas del producto.

De esta forma, cada producto puede representarse mediante un vector:

Producto 0 → [0.12, 0.73, 0.45, ..., 0.81]

Producto 1 → [0.15, 0.70, 0.41, ..., 0.79]

Producto 2 → [0.82, 0.11, 0.34, ..., 0.27]

...

Cuando un usuario consulta un producto, la empresa quiere encontrar rápidamente los productos más similares del catálogo. Para ello se calcula la distancia entre el vector correspondiente al producto consultado y cada uno de los vectores almacenados en el catálogo.

Ejercicio 9.1 - El alumno dispone de la versión original del algoritmo en "**relacion_productos.c**", así que para su optimización en este caso utilizaremos GPU Nvidia con CUDA como plataforma de procesamiento. Se le pide al alumnado que desarrolle una versión paralela de esta aplicación, así como la temporización de la versión secuencial y paralela.

EJERCICIO 10

Fuerza bruta PIN protegido con SHA-256

[Complejidad Computacional]

Un sistema almacena un PIN numérico de forma "protegida": en lugar de guardar el PIN en texto plano, guarda su hash SHA-256. Un hash es una función de un solo sentido: es fácil calcular el hash a partir del PIN, pero computacionalmente inviable (en general) recuperar el PIN a partir del hash sin más información.

Sin embargo, cuando el espacio de valores posibles es muy pequeño —como ocurre con un PIN de pocos dígitos— es posible romper la protección por fuerza bruta: basta con calcular el hash de todos los PINs posibles y compararlo con el hash almacenado, hasta encontrar una coincidencia.

En este ejercicio implementaremos ese ataque en C. Para ello, el alumno dispone de dos ficheros "**secreto4.txt**" y "**secreto8.txt**". Cada uno de ellos almacena el PIN de 4 dígitos y el de 8, respectivamente. Por otro lado, el alumno tiene disponible el programa "**crack.c**", este programa lee el PIN e implementa una serie de rutinas públicas para generar SHA-256. Con esto se pide al alumno que pruebe todas las combinaciones posibles, tal y como se muestra en el ejemplo del propio problema, hasta encontrar el PIN secreto.

Ejercicio 10.1- En primer lugar, el alumno se familiarizará con el código e implementará una rutina que encuentre el PIN secreto de 4 dígitos encriptado. Además, temporizará el proceso de fuerza bruta para reflexionar sobre la inseguridad de los PINs de 4 dígitos (recuerde que en la propia asignatura se habló de la seguridad en un CPD).

Ejercicio 10.2 - En segundo lugar, el alumno hará lo mismo para el PIN de 8 dígitos. En este caso, el alumno aplicará todos los conocimientos adquiridos para encontrar el PIN en el menor tiempo posible.